Rust for infrastructure engineers

Ownership and borrowing, errors with Result, structs enums and traits, CLI tools with clap and serde, async with tokio, and testing and shipping static binaries.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. Why Rust: ownership, borrowing and the mental model (13 min)
2. Errors: Option, Result and the question mark (12 min)
3. Structs, enums, traits and generics (12 min)
4. CLI tools with clap, serde and files (12 min)
5. Async Rust with Tokio and network services (13 min)
6. Testing, cross compilation and shipping static binaries (12 min)
Cheat sheet

Lesson 1. Why Rust: ownership, borrowing and the mental model

13 minutes. Understand why infrastructure teams choose Rust, and learn ownership, borrowing and lifetimes well enough to read the compiler errors instead of fighting them.

Rust gives the speed of C with memory safety checked at compile time. The price is a stricter way of thinking about who owns each value. Once that model clicks, the compiler becomes a reviewer.

What you will learn

- Where Rust fits in infrastructure work
- Ownership: one owner, moved on assignment
- Borrowing: shared and mutable references
- Lifetimes in plain terms
- Reading compiler errors

Where Rust fits

Rust suits tools that must be fast, small and safe: command line utilities, agents that run on every host, proxies, parsers and controllers. A single static binary with no runtime is easy to ship. Go or Python is often simpler for glue scripts, so choose Rust where its guarantees pay off.

Ownership

Every value has one owner, and the value is dropped when the owner goes out of scope. Assigning a String to another variable moves it, and the first variable can no longer be used. Simple types such as integers are copied instead. This rule removes use after free and double free bugs without a garbage collector.

Borrowing

You can lend a value with a reference. Any number of shared references `&T` may exist, or exactly one mutable reference `&mut T`, never both at once. This prevents data races and surprising changes while someone else reads the value.

Lifetimes

A reference must not outlive the value it points to. Usually the compiler infers this. When a function returns a reference, you may need a lifetime annotation such as `&'a str` to say which input it borrows from. If lifetimes feel hard, clone a value or return an owned String first and optimise later.

Compiler errors

Rust errors are detailed and often suggest the fix. Read the first error fully, since later errors are often caused by it. The common ones are use of a moved value, a borrow that lives too long and a mutable borrow while a shared one exists. Restructure to shorten borrows.

Diagram: Why Rust: ownership, borrowing and the mental model

Part	What it does	What goes wrong
Owner	Holds the value.	Moved away.
Borrow	Shared or mutable.	Conflicting borrows.
Compiler checks	Ownership rules.	Error to read.
Static binary	Safe and fast.	Large debug build.

Flow: Owner to Borrow / Borrow to Compiler checks / Compiler checks to Static binary

MARGA by EknathaLabs
marga.eknathalabs.com

Incident: The value used after it moved

Incident: *A first Rust tool failed to compile after a variable was passed to a function and used again.*

```
$ cargo build 2>&1 | head -6
error[E0382]: borrow of moved value: `config`
  --> src/main.rs:9:20
   |
 6 |     let config = load();
 8 |     run(config);
(this is a simulated illustration, not a real incident)
```

The function took ownership of config, so it could not be used afterwards. Passing a reference instead, `run(&config)`, kept the owner in main. The engineer learned to ask whether a function needs to own, borrow or mutate a value.

Common mistakes

- Cloning everything to silence the compiler
- Fighting the borrow checker instead of changing the design
- Ignoring the first error message
- Using unsafe to avoid a design problem
- Choosing Rust for a ten line script

Check yourself

How many owners does a Rust value have?

Answer: One

What may exist at the same time?

Answer: Many shared references or one mutable reference

What does moving a String do?

Answer: Transfers ownership so the old variable cannot be used

What is a simple fix for lifetime trouble?

Answer: Return an owned value

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. Errors: Option, Result and the question mark

12 minutes. Handle missing values and failures with Option and Result, propagate errors with the question mark operator and design error types for tools.

Rust has no exceptions and no null. Missing values and failures are ordinary values that the compiler makes you handle, which is why tools written in Rust rarely crash on bad input.

What you will learn

- Option for values that may be missing
- Result for operations that may fail
- The question mark operator
- Error types with thiserror and anyhow
- Panics versus errors

Option

`Option<T>` is either `Some(value)` or `None`. A map lookup returns an Option, because the key may be absent. You must say what to do for None, using `match`, `if let`, or helpers such as `unwrap_or` and `map`. There is no null pointer to forget.

Result

`Result<T, E>` is either `Ok(value)` or `Err(error)`. Reading a file returns a Result, because it may fail. The error type says why, for example the file was missing or permission was denied. Ignoring a Result gives a compiler warning.

The question mark

Inside a function that returns a Result, `?` after an expression returns early with the error if there was one, and otherwise gives the Ok value. It replaces long chains of match statements and keeps the happy path readable.

Error types

For libraries, define an error enum with the `thiserror` crate, so callers can match on the cause. For applications and command line tools, `anyhow::Result` carries any error and lets you add context with `.context("reading config")`. Good context turns a message such as No such file into a useful report.

Panics

A panic stops the thread and is for bugs, such as an impossible state. Do not use `unwrap` on input from users or files. In tools that run unattended, prefer returning errors and a clear exit code.

Diagram: Errors: Option, Result and the question mark

Part	What it does	What goes wrong
Input or file	May be missing.	Bad data.
Result	Ok or Err.	Ignored.
Question mark	Propagates errors.	No context.
Exit code and message	Clear failure.	Panic.

Flow: Input or file to Result / Result to Question mark / Question mark to Exit code and message

MARGA by EknathaLabs marga.eknathalabs.com

Incident: A panic at three in the morning

Incident: *A cleanup tool crashed on an unexpected file name and left a job half done.*

```
$ ./cleanup --dir /data/tmp
thread 'main' panicked at src/main.rs:31:46:
called `Result::unwrap()` on an `Err` value: Os { code: 13, kind: PermissionDenied }
$ echo $?
101
(this is a simulated illustration, not a real incident)
```

The code used unwrap on a file operation. It was changed to return a Result with context that names the path, the tool now reports the error and continues with the next file, and the exit code distinguishes partial failure from success.

Common mistakes

- unwrap on user input and files
- Errors with no context
- Using panic for expected failures
- Returning strings as errors
- Ignoring Results with let underscore

Check yourself

What is Option used for?

Answer: Values that may be missing

What does the question mark do?

Answer: Returns early with the error or gives the Ok value

Which crate gives easy application errors with context?

Answer: anyhow

When is a panic appropriate?

Answer: For a bug or impossible state

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. Structs, enums, traits and generics

12 minutes. Model data with structs and enums, share behaviour with traits and write generic code, so infrastructure tools express states clearly and safely.

Enums and traits are where Rust shines for infrastructure code. A state machine written as an enum cannot be in an impossible state, and traits let you swap implementations in tests.

What you will learn

- Structs and methods
- Enums that carry data and match
- Traits as shared behaviour
- Generics and trait bounds
- Derive macros for common behaviour

Structs

A struct groups named fields. Methods are written in an `impl` block, and they take `self`, `&self` or `&mut self` depending on whether they consume, read or change the value. Keep fields private and expose methods when you need to keep invariants.

Enums

Each variant of an enum may carry data. A deployment state such as `Pending`, `Running { pid: u32 }` or `Failed(String)` makes it impossible to read a pid for a failed deployment. A `match` must handle every variant, so adding a variant forces you to update every place that needs it.

Traits

A trait defines methods a type can implement, like an interface. A `Storage` trait with `put` and `get` can have a real implementation for S3 and an in-memory one for tests. Functions can take `&dyn Storage` for runtime choice or a generic parameter for compile time choice.

Generics and bounds

Generic functions work for many types: `fn largest<T: PartialOrd>(items: &[T])`. The bound says what the type must support. Generics are resolved at compile time and cost nothing at run time, but they can make error messages long. Start concrete and generalise when you have two uses.

Derive

`#[derive(Debug, Clone, PartialEq)]` generates common implementations. With serde, `#[derive(Serialize, Deserialize)]` turns a struct into JSON or YAML and back. This replaces a lot of parsing code and checks types at the boundary.

Diagram: Structs, enums, traits and generics

Part	What it does	What goes wrong
Structs and enums	Model state.	Impossible states.
Traits	Shared behaviour.	Over abstraction.
Generics	Reusable code.	Long errors.
Test doubles	Swap implementations.	No trait.

Flow: Structs and enums to Traits / Traits to Generics / Traits to Test doubles

MARGA by EknathaLabs marga.eknathalabs.com

Incident: A state that should not exist

> **Incident:** *A deploy tool reported a running service with no process id, because states were booleans and strings.*

```
$ grep -n "is_running" src/*.rs | head -3
src/deploy.rs:14:    pub is_running: bool,
src/deploy.rs:15:    pub pid: Option<u32>,
src/deploy.rs:41:    if d.is_running { println!("pid {}", d.pid.unwrap()); }
(this is a simulated illustration, not a real incident)
```

The struct allowed is_running true with pid None, and unwrap panicked. Replacing both fields with an enum whose Running variant carries the pid made that state impossible, and the compiler pointed to every place that needed an update.

Common mistakes

- Booleans and strings instead of enums
- Traits created before a second implementation exists
- Public fields that break invariants
- Deriving nothing and writing boilerplate
- Generic code that nobody can read

Check yourself

What can an enum variant carry?

Answer: Data

What does match require?

Answer: Handling every variant

What is a trait like?

Answer: An interface of shared behaviour

What does derive Serialize do?

Answer: Generates code to convert the type to formats such as JSON

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 4. CLI tools with clap, serde and files

12 minutes. Build a command line tool with clap for arguments, serde for configuration and JSON, and good exit codes and output for use in scripts.

A command line tool is the classic first Rust project for operations engineers. Clap and serde provide the hard parts, which leaves you free to focus on the task.

What you will learn

- Project layout and Cargo
- Arguments and subcommands with clap
- Reading JSON and YAML with serde
- Output formats and exit codes
- Testing a CLI

Layout

`cargo new tool` creates a binary crate. Keep `main.rs` small, put logic in a library module and add dependencies with `cargo add clap serde serde_json anyhow`. A library crate with a thin binary is easy to test.

Clap

With the derive feature, a struct describes the command line and clap generates parsing, help text and error messages. Subcommands are an enum, such as `check`, `apply` and `list`. Add `--output json` and `--dry-run` flags early, because operators expect them.

Serde

Define a struct for the configuration and derive Deserialize. Reading a file with `serde_yaml` or `serde_json` either gives a typed value or an error that names the field. Use `#[serde(deny_unknown_fields)]` to catch typos in config files, and defaults for optional fields.

Output and exit codes

Print results to standard output and diagnostics to standard error, so pipes work. Offer machine readable JSON for scripts. Exit with zero on success and distinct non zero codes for usage errors, partial failure and total failure, and document them.

Testing

Unit test the library functions with ordinary tests. For the binary use the `assert_cmd` crate, which runs it and checks output and exit code. Test the error paths, such as a missing file and invalid configuration, since that is where operators meet your tool.

Diagram: CLI tools with clap, serde and files

Part	What it does	What goes wrong
clap arguments	Subcommands and flags.	Bad usage.
serde config	Typed settings.	Typos.
Library logic	Testable core.	Logic in main.
Output and exit code	JSON and codes.	Always zero.

Flow: clap arguments to serde config / serde config to Library logic / Library logic to Output and exit code

Incident: A tool that always exited zero

Incident: *A CI check built in Rust passed every build because it exited with zero on errors.*

MARGA by EknathaLabs
marga.eknathalabs.com

```
$ ./check-config bad.yaml; echo "exit=$?"
error: unknown field `replicas_count`, expected one of `replicas`, `image`
exit=0
(this is a simulated illustration, not a real incident)
```

The main function printed the error and returned normally. Changing it to return a Result from main made the process exit with a non zero code. The team added a CLI test that asserts the exit code for invalid input.

Common mistakes

- Printing errors but exiting zero
- Mixing logs and results on standard output
- No dry run for changing commands
- Config files with no field validation
- Logic in main that cannot be tested

Check yourself

What does clap generate from a struct?

Answer: Parsing, help text and error messages

Why use deny_unknown_fields?

Answer: To catch typos in configuration

Where should diagnostics go?

Answer: Standard error

Which crate tests the compiled binary?

Answer: assert_cmd

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 5. Async Rust with Tokio and network services

13 minutes. Write concurrent network code with async and await on Tokio, use channels and timeouts, avoid blocking the runtime and build a small HTTP service.

Network tools such as proxies, exporters and agents spend most of their time waiting. Async Rust handles thousands of connections on a few threads, if you follow a few rules.

What you will learn

- Futures, async and await
- The Tokio runtime and tasks
- Timeouts, cancellation and select
- Channels and shared state
- Blocking calls and a small HTTP service

Futures

An `async fn` returns a future, a value that does nothing until it is awaited. `await` pauses the function until the result is ready and lets other tasks run. Futures are lazy, so forgetting to await a call means it never runs, and the compiler warns about it.

Tokio

Tokio provides a runtime that schedules tasks over a pool of threads. `#[tokio::main]` starts it, and `tokio::spawn` runs a future as a separate task. Tasks are cheap, so spawn one per connection. Use Tokio versions of file, network and timer operations.

Timeouts and cancellation

Wrap remote calls in `tokio::time::timeout`, because a network call without one can wait forever. Dropping a future cancels it, so be careful about work that must not stop halfway. `tokio::select!` waits for the first of several futures, for example a request or a shutdown signal.

Channels and state

Tasks communicate through channels such as `mpsc`, which avoids sharing memory. When you do share state, wrap it in `Arc<Mutex<T>>` and keep the lock short and never hold it across an await. For counters prefer atomics.

Blocking and HTTP

A blocking call such as a heavy computation or a synchronous file read stops a worker thread and starves other tasks. Run it with `spawn_blocking`. For HTTP, `axum` builds services from handler functions, and `reqwest` is a client. Add graceful shutdown so in-flight requests finish before exit.

Diagram: Async Rust with Tokio and network services

Part	What it does	What goes wrong
Incoming requests	Many connections.	Overload.
Tokio runtime	Schedules tasks.	Blocked threads.
Tasks and channels	Concurrent work.	Lock held across await.
Graceful shutdown	Finishes requests.	Abrupt exit.

Flow: Incoming requests to Tokio runtime / Tokio runtime to Tasks and channels / Tasks and channels to Graceful shutdown

MARGA by EknathaLabs
marga.eknathalabs.com

Incident: A service that froze under load

Incident: *An exporter stopped answering health checks when one handler read a large file.*

```
$ curl -s -m 2 localhost:9100/health; echo "exit=$?"
exit=28
$ grep -n "std::fs::read" src/*.rs
src/handlers.rs:22:    let data = std::fs::read(path).unwrap();
(this is a simulated illustration, not a real incident)
```

The handler used a blocking file read on the runtime thread, so with a few slow reads all workers were busy and health checks timed out. Using tokio::fs and spawn_blocking for the heavy parse restored responsiveness.

Common mistakes

- Blocking calls on the runtime
- No timeout on network calls
- Holding a mutex across an await
- Spawning unbounded tasks without limits
- No graceful shutdown

Check yourself

When does an async fn run?

Answer: When its future is awaited or spawned

What does spawn_blocking do?

Answer: Runs blocking work on a separate thread pool

What does select do?

Answer: Waits for the first of several futures

What is a good rule for mutexes?

Answer: Keep the lock short and not across an await

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. Testing, cross compilation and shipping static binaries

12 minutes. Test Rust code, run clippy and fmt in CI, cross compile to static musl binaries, keep images small and sign and publish releases.

A big advantage of Rust is the artifact: one small binary with no runtime. Testing and release engineering make sure that artifact is trustworthy and easy to deploy everywhere.

What you will learn

- Unit, integration and doc tests
- Clippy, fmt and cargo audit in CI
- Cross compiling and static linking with musl
- Release profiles and binary size
- Containers, signing and provenance

Tests

Unit tests live next to the code in a `#[cfg(test)]` module and integration tests in the `tests` directory. Doc tests run the examples in comments. `cargo test` runs all of them, and property testing with `proptest` finds edge cases you did not think of.

CI checks

Run `cargo fmt --check`, `cargo clippy -- -D warnings` and `cargo test` on every change. `cargo audit` checks dependencies against the advisory database, and `cargo deny` can enforce licence and source rules. Cache the build directory to keep CI fast.

Cross compiling

Add a target with `rustup target add x86_64-unknown-linux-musl` and build with `--target`. The musl target links statically, so the binary runs on any Linux without a matching libc. Tools such as `cross` or `cargo zigbuild` handle linkers and C dependencies for other platforms.

Release profile

Set `opt-level`, `lto = true`, `codegen-units = 1` and `strip = true` in the release profile to shrink and speed up the binary at the cost of build time. Measure the size, since a few megabytes is normal and tens of megabytes means unneeded dependencies.

Containers and signing

Copy the static binary into a scratch or distroless image for a very small container with little to attack. Generate an SBOM, sign the artifacts and publish checksums. Pin the Rust toolchain version in `rust-toolchain.toml` so builds are repeatable.

Diagram: Testing, cross compilation and shipping static binaries

Part	What it does	What goes wrong
cargo test and clippy	Quality gate.	Skipped.
Release build	LTO, strip.	Slow build.
Static musl target	No libc needed.	C dependencies.
Scratch image and signature	Small and verifiable.	Unsigned.

Flow: cargo test and clippy to Release build / Release build to Static musl target / Static musl target to Scratch image and signature

MARGA by EknathaLabs marga.eknathalabs.com

Incident: A binary that would not start on the server

Incident: *A tool built on a developer laptop failed on an older server with a missing library version.*

```
$ ./agent --version
./agent: /lib64/libc.so.6: version `GLIBC_2.34' not found (required by ./agent)
$ ldd ./agent | head -3
    linux-vdso.so.1
    libc.so.6 => /lib64/libc.so.6
(this is a simulated illustration, not a real incident)
```

The binary was linked dynamically against a newer libc than the server had. Building for the musl target in CI produced a static binary that ran everywhere, and the pipeline now tests the artifact in a container with an old distribution.

Common mistakes

- Building releases on laptops
- Dynamic linking to a newer libc
- No dependency audit
- Unpinned toolchain
- Huge binaries from unused features

Check yourself

What does the musl target give?

Answer: A statically linked binary

What does cargo audit check?

Answer: Dependencies against known advisories

Which flag treats clippy warnings as errors?

Answer: -D warnings

Why pin the toolchain?

Answer: Repeatable builds

MARGA by EknathaLabs
marga.eknathalabs.com

Cheat sheet: Rust for infrastructure

Cargo commands, static builds and error handling choices.

Cargo

Job	Command
New project	`cargo new tool`
Build release	`cargo build --release`
Test	`cargo test`
Lint	`cargo clippy -- -D warnings`
Format	`cargo fmt`

Static builds

Step	Command
Add target	`rustup target add x86_64-unknown-linux-musl`
Build	`cargo build --release --target x86_64-unknown-linux-musl`
Check	`ldd target/.../tool` should say not a dynamic executable

Errors

Need	Use
Application	`anyhow::Result` and `?`
Library	`thiserror` enum
Missing value	`Option` and `ok_or`

MARGA by EknathaLabs
marga.eknathalabs.com